CS6886W Assignment 1: VGG6 on CIFAR-10

GitHub repository: https://github.com/cs25m545-debug/SystemLearningAssignment1

1. Baseline and data preparation

The linked template topology is preserved: 3x3 convolutions with channels 64, 64, 128, 128; batch normalization and the chosen activation after every convolution; 2x2 max pooling after each pair; adaptive global average pooling; a 128-to-10 linear classifier. Cross-entropy consumes logits.

CIFAR-10 training data is split into 45000 training and 5000 validation examples by a fixed seeded permutation. The official 10,000-example test set is never used during sweeps. Training transforms: random 32x32 crop with padding 4, random horizontal flip (p=0.5), tensor conversion, normalization with mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010), then one clipped cutout square of side 16 (zero disables cutout). Validation and test use tensor conversion and the same normalization only. Training accuracy is measured on augmented minibatches before each update; validation accuracy uses eval mode on unaugmented held-out data.

Selected checkpoint test top-1 accuracy: 86.6100%. Test loss: 0.408060; evaluated samples: 10000.

2. Configuration comparisons

The baseline is shared across one-factor comparisons. Joint-search runs are excluded from causal one-factor conclusions but remain eligible for final selection. Accuracies are percentages. t60 is the first wall-clock time reaching 60% validation accuracy; '-' means the threshold was never reached.

Activation

ReLU has a nonsaturating positive branch but can have inactive units. Sigmoid saturates and is not zero-centered; tanh is zero-centered but also saturates. SiLU and GELU are smooth gates that preserve small negative outputs. Batch normalization and the shared template initialization are held fixed, so this comparison does not tune initialization independently for each activation.

Value	Best val %	Final val %	Best epoch	Steps	Seconds	t60
relu	87.740	87.740	100	35200	794.3	32.3
sigmoid	71.680	71.300	93	35200	775.9	636.8
tanh	79.900	79.620	90	35200	778.8	92.8
silu	86.960	86.520	90	35200	781.2	39.1
gelu	87.500	87.200	87	35200	782.9	23.5

The strongest observed activation value in this controlled comparison is relu, at 87.740% best validation accuracy. Its difference from the baseline is +0.000 percentage points. This conclusion applies only to the measured configuration budget.

Optimizer

SGD uses momentum 0.9; Nesterov-SGD uses its look-ahead correction. Adam combines first and second moment estimates; Nadam adds Nesterov-style momentum to Adam. Adagrad accumulates squared gradients, which can shrink its effective step size over long runs. RMSprop uses an exponential squared-gradient average rather than an ever-growing accumulator. Defaults are PyTorch defaults except learning rate, weight decay, and SGD momentum. A shared learning

rate isolates the optimizer choice but is not each optimizer's individually tuned optimum.

Value	Best val %	Final val %	Best epoch	Steps	Seconds	t60
adam	87.740	87.740	100	35200	794.3	32.3
sgd	74.660	74.200	87	35200	763.2	122.1
nesterov	74.980	74.660	98	35200	765.7	107.3
adagrad	62.480	62.220	79	35200	769.0	230.1
rmsprop	88.080	87.760	92	35200	770.8	92.8
nadam	87.980	87.720	99	35200	778.4	46.8

The strongest observed optimizer value in this controlled comparison is rmsprop, at 88.080% best validation accuracy. Its difference from the baseline is +0.340 percentage points. This conclusion applies only to the measured configuration budget.

Batch Size

Smaller batches give more optimizer updates per epoch and noisier gradients. Larger batches can improve hardware throughput but give fewer updates per epoch. Compare both elapsed time and step counts, not epochs alone; no linear learning-rate scaling is applied in this controlled comparison.

Value	Best val %	Final val %	Best epoch	Steps	Seconds	t60
128	87.740	87.740	100	35200	794.3	32.3
64	87.820	87.820	100	70400	942.9	28.7
256	86.920	86.820	98	17600	749.0	38.5

The strongest observed batch_size value in this controlled comparison is 64, at 87.820% best validation accuracy. Its difference from the baseline is +0.080 percentage points. This conclusion applies only to the measured configuration budget.

Epochs

Longer runs give more optimization opportunities but may overfit. Best and final validation accuracies expose late degradation. With cosine scheduling, changing the epoch budget also changes the schedule horizon; these are budget comparisons, not identical learning-rate trajectories truncated at different epochs.

Value	Best val %	Final val %	Best epoch	Steps	Seconds	t60
100	87.740	87.740	100	35200	794.3	32.3
25	82.520	82.400	24	8800	194.6	31.2
50	86.080	86.060	49	17600	388.5	30.9

The strongest observed epochs value in this controlled comparison is 100, at 87.740% best validation accuracy. Its difference from the baseline is +0.000 percentage points. This conclusion applies only to the measured configuration budget.

Learning Rate

Very small learning rates can converge slowly; large rates can oscillate or diverge. The initial rate is varied while cosine scheduling, optimizer, seed, and split remain fixed. Observed differences below are single-seed measurements, not evidence of statistical significance.

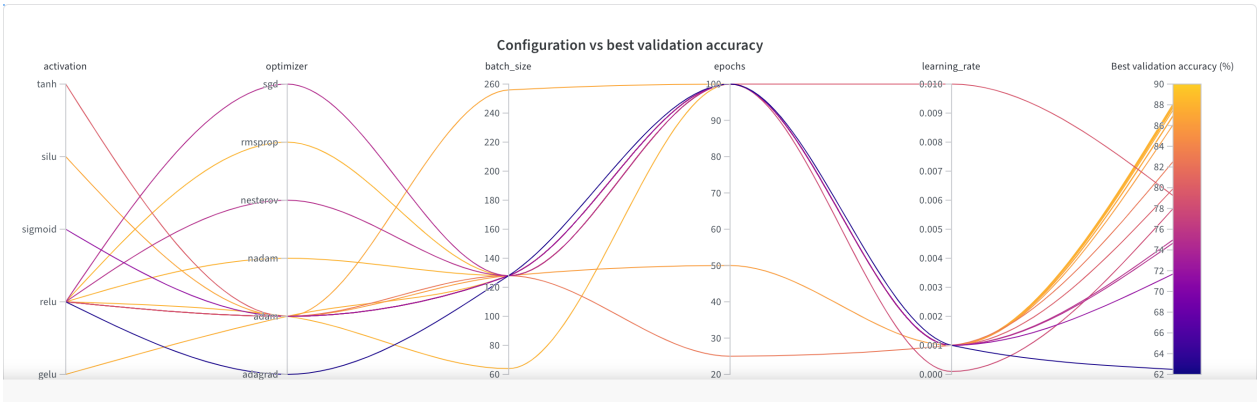
Value	Best val %	Final val %	Best epoch	Steps	Seconds	t60
0.001	87.740	87.740	100	35200	794.3	32.3
0.0001	77.980	77.520	85	35200	777.8	62.3
0.01	79.240	78.840	98	35200	776.2	85.4

The strongest observed learning_rate value in this controlled comparison is 0.001, at 87.740% best validation accuracy. Its difference from the baseline is +0.000 percentage points. This conclusion applies only to the measured configuration budget.

3. Plots

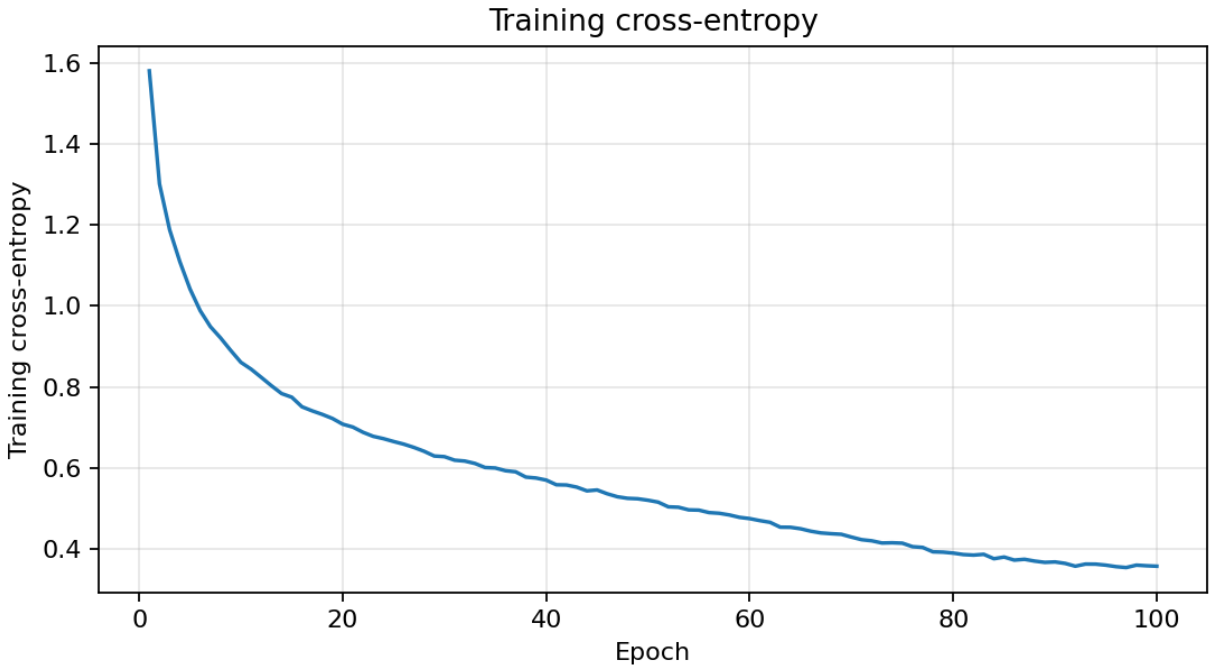
W&B report: https://wandb.ai/cs25m545-indian-institute-of-technology-madras/vgg6-cifar10/reports/CS6886W-Assignment-1---Verified-VGG6-experiments--VmlldzoxODIzNzMyMw==

W&B parallel-coordinate panel exported from the experiment workspace.

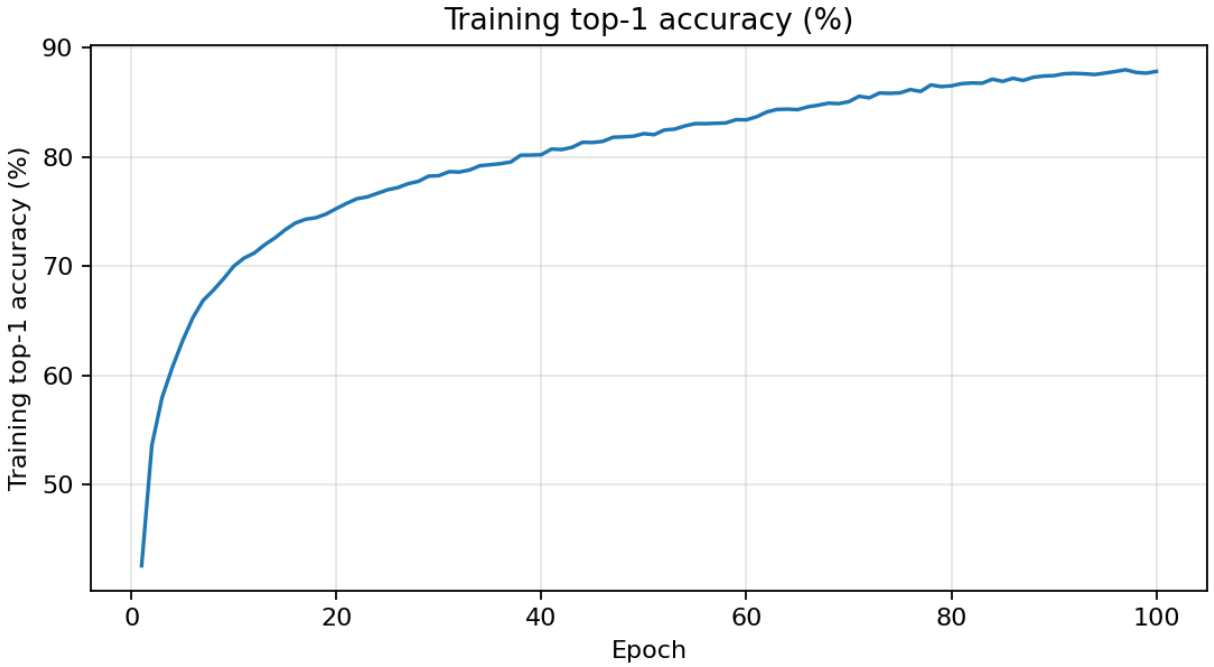


The following selected-run plots are generated automatically and logged as W&B images by the training program. Native W&B scalar panels also use the same logged history with optimizer update step as their x-axis.

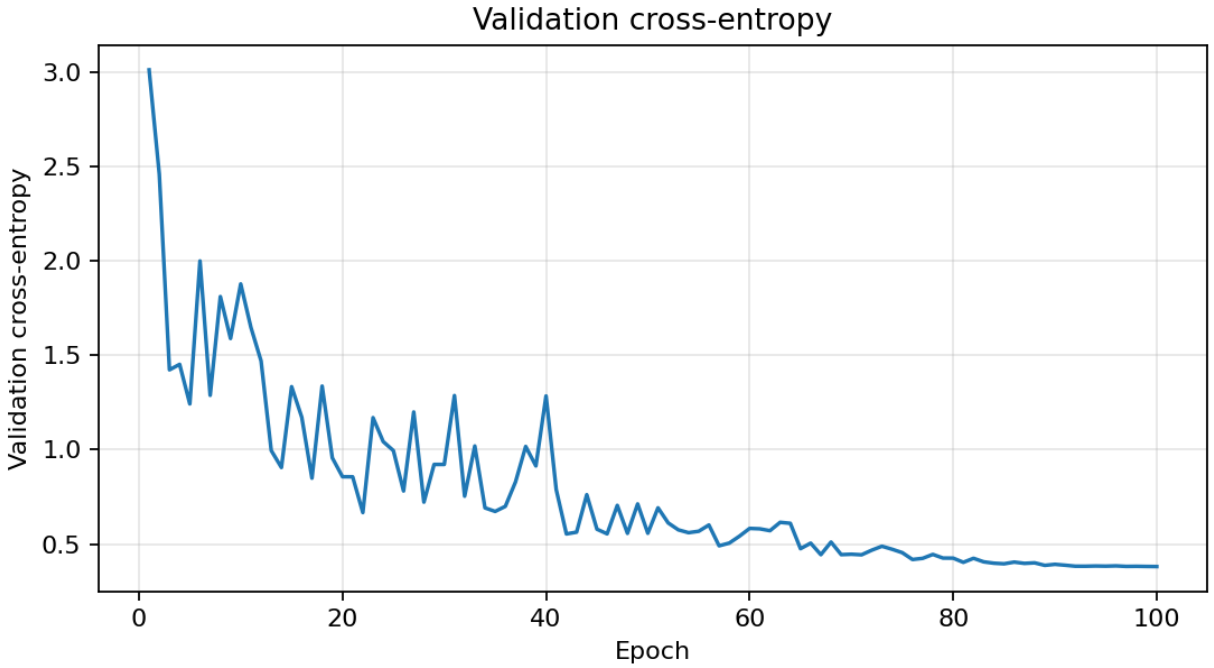
Training Loss



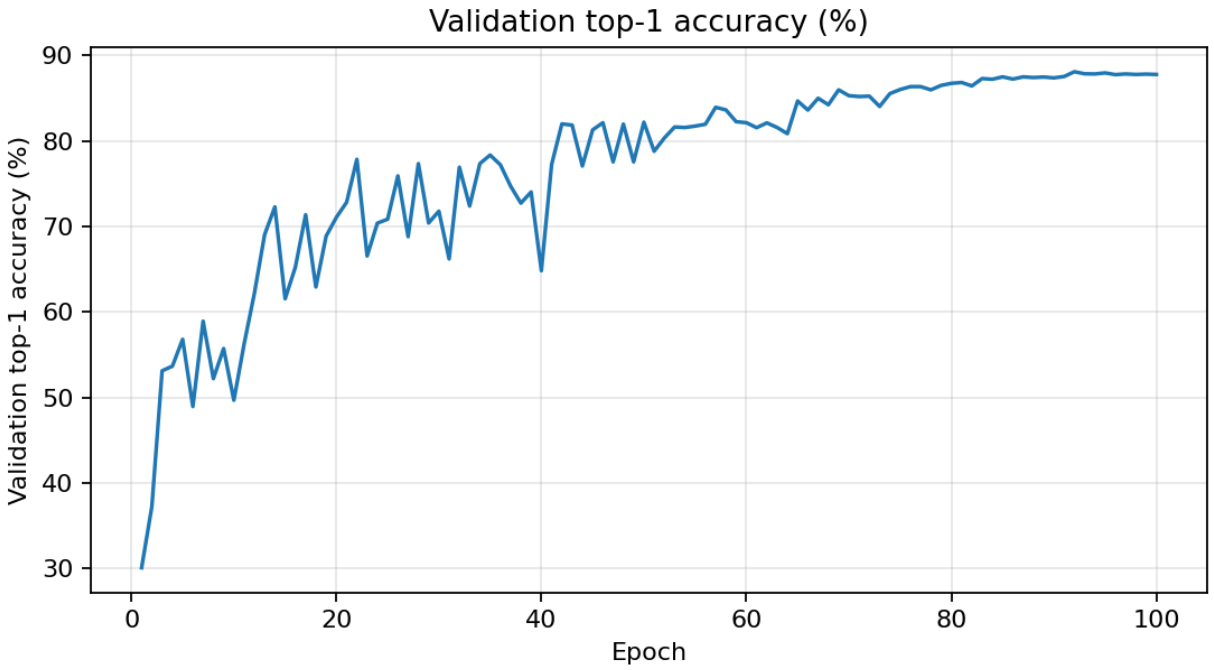
Training Accuracy



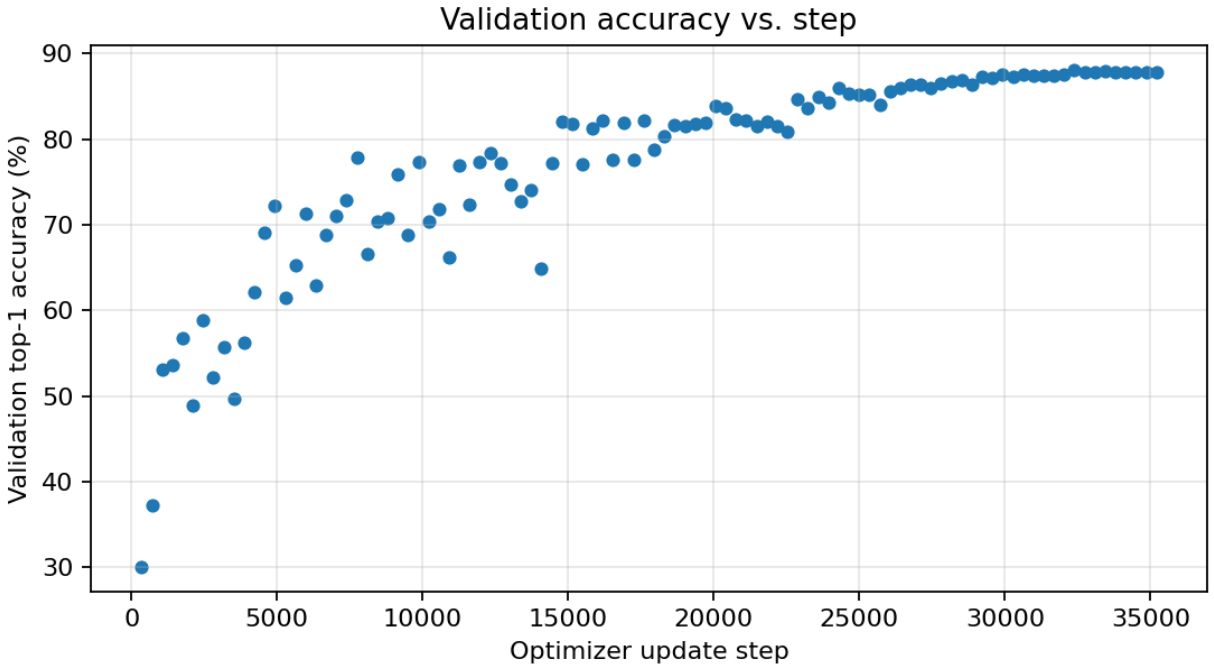
Validation Loss



Validation Accuracy



Validation Scatter



4. One final configuration

Exactly one winner was selected by highest validation accuracy; ties prefer the earlier best epoch, then sorted run path. Best validation accuracy: 88.0800%; checkpoint epoch: 92. The best checkpoint, not the last epoch, was evaluated on the test set.

Parameter	Value
activation	relu
optimizer	rmsprop

Parameter	Value
batch_size	128
epochs	100
learning_rate	0.001
weight_decay	0.0005
momentum	0.9
seed	42
split_seed	42
validation_size	5000
workers	0
device	mps
data_dir	data
scheduler	cosine
cutout	16
synthetic	False
group	optimizer

Selected W&B run:
https://wandb.ai/cs25m545-indian-institute-of-technology-madras/vgg6-cifar10/runs/x0hu8nqy

5. Reproducibility and repository

See README.md for exact installation, sweep, selection, evaluation, and PDF commands. Python, NumPy, PyTorch, shuffle generators, and worker RNGs are seeded. Deterministic algorithms are enabled and cuDNN benchmarking is disabled. Identical software and hardware are required for bitwise reproduction; cross-device exact accuracy is not guaranteed. uv.lock pins transitive dependencies.

An independent from-scratch rerun on the same Mac reproduced validation accuracy 88.0800% and test accuracy 86.6100%, selecting epoch 92. The verification is saved in artifacts/mac-final/reproduction.json.

Environment	Value
python	3.11.14
platform	macOS-26.6.2-arm64-arm-64bit
torch	2.6.0
torchvision	0.21.0
numpy	2.2.3
device	mps
cuda	None
gpu	None
deterministic_algorithms	True

The trained checkpoint is artifacts/mac-final/best.pt. Its configuration, results, history, and environment are saved alongside it. The saved model is a state dictionary with its configuration; evaluation reconstructs the architecture without pickled model objects. Confirm the repository and checkpoint are accessible to evaluators.